Practical tricks notebook

QUICK INDEX + SHARED SETUP

Technique	Contract	Page
1. Berlekamp-Massey + nth term	Exact (recurrence)	2
2. Tutte matrix matching size	Bounded Error	3
3. Vertex cover with a small budget	Exact	4
4. Color-coding for short simple paths	Bounded Error	5
5. Bit-parallel LCS length	Exact	6
6. Algorithm X with bitmasks	Exact	7
7. Posa rotations for Hamiltonian paths	Heuristic	8
8. Shuffled window subset sum	Heuristic	9
9. Schoning randomized 3-SAT	Bounded Error	10
10. Consecutive-point Lagrange interpolation	Exact (polynomial)	11
11. General Lagrange interpolation	Exact (polynomial)	12
12. Connected subsets with one centroid	Exact	13

Copy once. Use this setup before the selected snippets. Local includes are omitted in print. Do not use `#define int long long`. Arithmetic uses long long; indices use int; bit operations use unsigned 64-bit words.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
const ll HMOD = 1000000007;
ll hnorm(ll x) { x %= HMOD; return x < 0 ? x + HMOD : x; }
ll hpow(ll a, ll e) {
    ll r = 1; a = hnorm(a);
    for (; e; e >>= 1, a = a * a % HMOD)
        if (e & 1) r = r * a % HMOD;
    return r;
}
// Seed once in main; use a fixed seed while debugging.
// mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ll pick(mt19937_64& rng, ll lo, ll hi) {
    return uniform_int_distribution<ll>(lo, hi)(rng);
}
```

RNG. Seed once in main, then pass rng by reference. Keep a fixed seed for debugging; use the printed runtime seed for submission. Modular power costs O(log e); normalization and random sampling cost O(1).

Read the contract. Exact searches may take exponential time. A heuristic's nullopt/false means NOT FOUND, not impossible. Probabilistic bounds assume independent uniform samples on a fixed input; sum failure bounds across calls.

Validate. Run `bash tools/test_hack_notebook.sh`; add `SANITIZE=1` for address/undefined-behavior checks. Fixed-seed tests do not prove reliability on every input. Check witnesses and benchmark your problem's constraints.

Rebuild. `python3 tools/build_hack_notebook.py` requires reportlab. The PDF reads actual tested snippets and the notes in hack_notebook.md.

1. Berlekamp-Massey + nth term

EXACT UNDER RECURRENCE CONTRACT

Purpose. Count walks or finite-state processes at a huge index. Generate initial answers with a slow DP, recover their recurrence, then jump to N.

Idea. BM repairs a recurrence when the next sample disagrees. To evaluate it, compute x^N modulo its characteristic polynomial. A fixed linear transition on S states guarantees recurrence order at most S, so 2S samples suffice over this field.

```
vector<ll> berlekampMassey(vector<ll> s) {
    for (ll& x : s) x = hnorm(x);
    vector<ll> C{1}, B{1};
    int L = 0, shift = 1; ll last = 1;
    for (int i = 0; i < (int)s.size(); ++i) {
        ll d = s[i];
        for (int j = 1; j <= L; ++j)
            d = (d + C[j] * s[i-j]) % HMOD;
        if (!d) { ++shift; continue; }
        auto old = C;
        ll coef = d * hpow(last, HMOD-2) % HMOD;
        C.resize(max(C.size(), B.size()+shift), 0);
        for (int j = 0; j < (int)B.size(); ++j)
            C[j+shift] = hnorm(C[j+shift] - coef*B[j]%HMOD);
        if (2*L <= i) {
            L = i+1-L; B = old; last = d; shift = 1;
        } else ++shift;
    }
    C.resize(L+1); C.erase(C.begin());
    for (ll& x : C) x = hnorm(-x);
    return C; // s[i] = sum rec[j] * s[i-1-j]
}
ll nthTerm(vector<ll> initial, vector<ll> rec, ull N) {
    int L = rec.size(); assert(initial.size() >= rec.size());
    for (ll& x : initial) x = hnorm(x);
    for (ll& x : rec) x = hnorm(x);
    if (N < initial.size()) return initial[N];
    if (!L) return 0;
    auto mul = [&](vector<ll> a, vector<ll> b) {
        vector<ll> c(2*L-1);
        for (int i = 0; i < L; ++i)
            for (int j = 0; j < L; ++j)
                c[i+j] = (c[i+j] + a[i]*b[j]) % HMOD;
        for (int i = 2*L-2; i >= L; --i)
            for (int j = 1; j <= L; ++j)
                c[i-j] = (c[i-j] + c[i]*rec[j-1]) % HMOD;
        c.resize(L); return c;
    };
    vector<ll> ans(L), x(L); ans[0] = 1;
    if (L == 1) x[0] = rec[0]; else x[1] = 1;
    for (; N; N >>= 1, x = mul(x, x))
        if (N & 1) ans = mul(ans, x);
    ll value = 0;
    for (int i = 0; i < L; ++i)
        value = (value + ans[i]*initial[i]) % HMOD;
    return value;
}
```

Complexity. For s samples: O(s^2 + s log P). For order L: O(L^2 log N) evaluation. O(s + L) memory, plus the cost of generating samples. P = 1000000007.

Pitfalls. A recurrence fitting a finite prefix need not predict an arbitrary sequence. Prove an order bound; use at least twice that many samples. Supply at least L initial terms. Empty recurrence denotes the zero sequence. This is modular arithmetic, not integer sequence reconstruction.

Example. `auto r = berlekampMassey({0,1,1,2});` then `nthTerm({0,1}, r, 10) == 55`. N is 0-indexed.

Tests. Empty/zero sequences, order one, leading zeros, negative coefficients, 1,000 generated recurrences with held-out terms, and huge unsigned indices checked against matrix exponentiation.

Sources. KACTL BM; KACTL recurrence evaluation.

2. Tutte matrix matching size

BOUNDED ERROR / LOWER BOUND

Purpose. Find the size of a maximum matching in a general undirected graph, including graphs with odd cycles. This version does not recover matching edges.

Idea. Assign each edge an independent random field value r; put r and -r in opposite matrix entries. The symbolic rank is twice the matching size. Random evaluation preserves that rank with high probability; it can only lower it.

```
int tutteSize(int n, const vector<pair<int,int>>& edges,
              int trials, mt19937_64& rng) {
    assert(n >= 0 && n < HMOD && trials > 0);
    int best = 0;
    while (trials--) {
        vector<vector<ll>> a(n, vector<ll>(n));
        for (auto [u,v] : edges) {
            assert(0 <= u && u < n && 0 <= v && v < n && u != v);
            ll r = pick(rng, 0, HMOD-1);
            a[u][v] = r; a[v][u] = hnorm(-r);
        }
        int rank = 0;
        for (int col = 0; col < n; ++col) {
            int row = rank;
            while (row < n && !a[row][col]) ++row;
            if (row == n) continue;
            swap(a[row], a[rank]);
            ll inv = hpow(a[rank][col], HMOD-2);
            for (int i = rank+1; i < n; ++i) {
                ll f = a[i][col]*inv % HMOD;
                for (int j = col; j < n; ++j)
                    a[i][j] = hnorm(a[i][j] - f*a[rank][j]%HMOD);
            }
            ++rank;
        }
        assert(rank % 2 == 0);
        best = max(best, rank/2);
    }
    return best;
}
```

Complexity. O(t n^3) time, O(n^2) memory for t trials. For n < P, failure is at most (n/P)^t; take the maximum result across trials.

Pitfalls. Prime P must be odd. Input has vertices 0..n-1 and no self-loops; duplicate edges are harmless here. A result is a lower bound, not a certificate of optimality. Never replace random edge weights by all ones.

Example. `tutteSize(3, {{0,1},{1,2},{2,0}}, 3, rng)` returns 1 with high probability.

Tests. Every simple graph through 5 vertices versus exhaustive matching; 300 random graphs through 12 vertices; empty graphs, odd cycles, duplicate input edges and disconnected components.

Sources. Randomized general matching with Tutte matrix.

3. Vertex cover with a small budget

EXACT / EXPONENTIAL

Purpose. Delete at most k vertices so no conflicting pair remains. Useful when k is small even if n is large. Returns selected vertices, or nullopt if impossible.

Idea. Every uncovered edge needs one endpoint selected. Branch on its endpoints. If a vertex has more than k remaining neighbors, selecting it is forced: excluding it would require selecting all those neighbors.

```
optional<vector<int>> vertexCover(int n, vector<pair<int,int>> edges, int k) {
    vector<int> chosen;
    function<bool(const vector<pair<int,int>>&,int)> dfs =
    [&](const vector<pair<int,int>>& e, int left) {
        if (left < 0) return false;
        if (e.empty()) return true;
        if (!left) return false;
        vector<int> degree(n);
        for (auto [u,v] : e) { ++degree[u]; ++degree[v]; }
        int forced = max_element(degree.begin(), degree.end())-degree.begin();
        vector<int> branch{e[0].first, e[0].second};
        if (degree[forced] > left) branch = {forced};
        for (int v : branch) {
            vector<pair<int,int>> next;
            for (auto [a,b] : e)
                if (a != v && b != v) next.push_back({a,b});
            chosen.push_back(v);
            if (dfs(next, left-1)) return true;
            chosen.pop_back();
        }
        return false;
    };
    if (dfs(edges, k)) return chosen;
    return nullopt;
}
```

Complexity. O(2^k (n+m)) time. O((k+1)(n+m)) memory for recursive edge-list copies. Recursion depth is at most k+1.

Pitfalls. The graph must be simple and undirected, with valid vertices and no self-loops or duplicate edges. Duplicates break the degree-based forced rule. The result is a cover of size at most k; it is not necessarily minimum. Try budgets in increasing order if a minimum is required.

Example. `vertexCover(3, {{0,1},{1,2}}, 1)` returns `{1}`. A present empty vector is a valid solution for an edgeless graph.

Tests. Every graph through 5 vertices at all budgets; random graphs versus subset enumeration; k=0, negative budget, isolated vertices, a 100-vertex star, and repeated calls.

Sources. MIT parameterized algorithms notes.

4. Color-coding for short simple paths

BOUNDED ERROR / VERIFIED WITNESS

Purpose. Find a simple path containing exactly k vertices in a directed or undirected graph. Returns the vertex sequence; nullopt means not found.

Idea. Randomly assign k colors. Subset DP tracks paths with distinct colors, which automatically have distinct vertices. A fixed target path is colorful with probability p = k!/k^k per trial.

```
optional<vector<int>> colorfulPath(const vector<vector<int>>& g,
                                   int k, int trials, mt19937_64& rng) {
    int n = g.size(); assert(0 <= k && k <= 15 && trials >= 0);
    if (!k) return vector<int>{};
    if (k > n) return nullopt;
    int masks = 1 << k;
    while (trials--) {
        vector<int> color(n);
        for (int& c : color) c = pick(rng, 0, k-1);
        vector<vector<int>> par(masks, vector<int>(n, -2));
        for (int v = 0; v < n; ++v) par[1<<color[v]][v] = -1;
        for (int mask = 1; mask < masks; ++mask)
            for (int v = 0; v < n; ++v) if (par[mask][v] != -2)
                for (int u : g[v]) if (!(mask >> color[u] & 1))
                    par[mask | (1<<color[u])][u] = v;
        for (int v = 0; v < n; ++v) if (par[masks-1][v] != -2) {
            vector<int> path; int mask = masks-1;
            while (v != -1) {
                path.push_back(v); int u = par[mask][v];
                mask ^= 1 << color[v]; v = u;
            }
            reverse(path.begin(), path.end()); return path;
        }
    }
    return nullopt;
}
```

Complexity. O(t 2^k (n+m)) time, O(2^k n) memory. Miss probability is at most (1-p)^t. The implementation caps k at 15, but practical limits are often far smaller.

Pitfalls. k counts vertices, not edges. Budget both the DP memory and the repetitions: p is about 0.0384 for k=5 and 0.000363 for k=10. A failed run does not establish impossibility. Every adjacency index must be valid.

Example. `colorfulPath({{1},{2},{}}, 3, 100, rng)` searches for `{0,1,2}`. A request for k=0 returns the empty path.

Tests. Small directed graphs versus exhaustive simple-path search, reconstructed edge/uniqueness checks, empty input, k>n, k=1 and zero trials. Fixed seeds make regressions repeatable.

Sources. Alon, Yuster, Zwick: Color-coding.

5. Bit-parallel LCS length

EXACT

Purpose. Compute the longest common subsequence length of two byte strings using word-level parallelism. It does not reconstruct the subsequence.

Idea. Adjacent DP values differ by at most one. Store those differences as bits. For a character mask M, use X = S | M, Y = (S << 1) | 1, then S = X & ~(X-Y). The answer is popcount(S).

```
int bitLCS(const string& a, const string& b) {
    int words = (b.size()+63)/64;
    vector<vector<ull>> match(256, vector<ull>(words));
    for (int j = 0; j < (int)b.size(); ++j)
        match[(unsigned char)b[j]][j/64] |= 1ULL << (j%64);
    vector<ull> s(words);
    for (unsigned char c : a) {
        ull carry = 1, borrow = 0;
        for (int j = 0; j < words; ++j) {
            ull old = s[j], x = old | match[c][j];
            ull y = (old << 1) | carry;
            carry = old >> 63;
            ull z = x-y, next = (x < y) | (z < borrow);
            z -= borrow; borrow = next;
            s[j] = x & ~z;
        }
    }
    int ans = 0;
    for (ull x : s) ans += __builtin_popcountll(x);
    return ans;
}
```

Complexity. Let w = ceil(|b|/64). Time O((|a|+256)w + |b|), memory O(256w). Use the shorter string as b to reduce mask storage. String lengths and the answer must fit int.

Pitfalls. Unsigned wraparound is deliberate. Carry from shifts and borrow from subtraction must cross word boundaries. Do not use signed words. Character masks use unsigned byte indices, including bytes 128..255. This is subsequence, not substring.

Example. `bitLCS("banana", "ananas") == 5`; `bitLCS("abc", "ac") == 2`.

Tests. Empty strings, all equal/disjoint characters, lengths 63/64/65 and 127/128/129, embedded zero/high bytes, and 1,500 random comparisons against ordinary DP.

Sources. Allison and Dix: A bit-string longest-common-subsequence algorithm.

6. Algorithm X with bitmasks

EXACT / EXPONENTIAL

Purpose. Select rows that cover every required column exactly once: tilings, small puzzles, or compatible configurations. Returns original row indices, or nullopt if impossible.

Idea. Pick the uncovered column with the fewest currently compatible rows. Try each of those rows, recurse, and undo the choice. Bitmasks give a short Algorithm X implementation; full Dancing Links is omitted to save space.

```
optional<vector<int>> exactCover(int columns, const vector<ull>& rows) {
    assert(0 <= columns && columns <= 64);
    ull full = columns == 64 ? ~0ULL : (1ULL << columns)-1;
    for (ull r : rows) assert(!(r & ~full));
    vector<int> chosen;
    function<bool(ull)> dfs = [&](ull covered) {
        if (covered == full) return true;
        int best = -1, count = INT_MAX;
        for (int c = 0; c < columns; ++c) if (!(covered >> c & 1)) {
            int ways = 0;
            for (ull r : rows)
                ways += !(r & covered) && (r >> c & 1);
            if (ways < count) { count = ways; best = c; }
        }
        if (!count) return false;
        for (int i = 0; i < (int)rows.size(); ++i)
            if (!(rows[i] & covered) && (rows[i] >> best & 1)) {
                chosen.push_back(i);
                if (dfs(covered | rows[i])) return true;
                chosen.pop_back();
            }
        return false;
    };
    if (dfs(0)) return chosen;
    return nullopt;
}
```

Complexity. Exponential search; O(c r) work per node for c columns and r rows. O(c) extra stack/solution space, apart from input. At most 64 columns.

Pitfalls. Row bits must lie within the declared columns. Every column is mandatory and must be covered exactly once; this is not ordinary set cover. Zero rows are ignored. Column 63 is supported; avoid evaluating 1ULL<<64. A present empty vector solves zero columns.

Example. `exactCover(3, {3,4,5})` returns indices `{0,1}`: row masks 011 and 100 cover 111 without overlap.

Tests. Zero columns, impossible columns, duplicate/empty rows, multiple solutions, bit 63, all 64 singleton columns, and 1,000 random instances versus row-subset enumeration.

Sources. Knuth: Dancing links.

7. Posa rotations for Hamiltonian paths

HEURISTIC / VERIFIED WITNESS

Purpose. Search for a path visiting every vertex exactly once in an undirected graph. Useful for constructive tasks where a found path can be checked. Returns nullopt when the search budget is exhausted.

Idea. Extend to a random unused neighbor. When stuck, if the last vertex joins path[i], reverse the suffix after i. The same vertices form a valid path with a new endpoint, which may allow further extension.

```
optional<vector<int>> posaPath(const vector<vector<int>>& g,
                               int restarts, int steps, mt19937_64& rng) {
    int n = g.size(); assert(restarts >= 0 && steps >= 0);
    if (!n) return vector<int>{};
    while (restarts--) {
        vector<int> path{(int)pick(rng, 0, n-1)}, pos(n, -1);
        pos[path[0]] = 0;
        for (int it = 0; ; ++it) {
            if ((int)path.size() == n) return path;
            if (it == steps) break;
            int last = path.back(); vector<int> next, pivots;
            for (int u : g[last]) {
                if (pos[u] == -1) next.push_back(u);
                else if (pos[u]+2 < (int)path.size())
                    pivots.push_back(pos[u]);
            }
            if (!next.empty()) {
                int u = next[pick(rng, 0, next.size()-1)];
                pos[u] = path.size(); path.push_back(u);
            } else if (!pivots.empty()) {
                int p = pivots[pick(rng, 0, pivots.size()-1)];
                reverse(path.begin()+p+1, path.end());
                for (int i = p+1; i < (int)path.size(); ++i) pos[path[i]] = i;
            } else break;
        }
    }
    return nullopt;
}
```

Complexity. O(R (B+1) n) time for R restarts and B extension/rotation steps per restart, on a simple graph. O(n) extra memory. No general success guarantee.

Pitfalls. Adjacency must be symmetric with valid indices, no loops or duplicates. Suffix reversal is invalid in general directed graphs. It finds a path, not necessarily a cycle: check the closing edge separately. Failure is NOT FOUND, not NO.

Example. `posaPath({{1},{0,2},{1}}, 20, 100, rng)` searches for `{0,1,2}` or its reverse. Verify n distinct vertices and every consecutive edge.

Tests. Exhaustive small undirected graphs checked by brute-force Hamiltonian search; every returned witness checked; planted paths in larger graphs; empty/singleton/disconnected inputs and zero-step budgets.

Sources. Ruskey: Combinatorial Generation, section 5.13.

8. Shuffled window subset sum

HEURISTIC / ONE-SIDED DECISION

Purpose. Search for a subset of nonnegative weights summing exactly to target when full target-sized DP is too large. true proves reachability; false means NOT FOUND. No subset reconstruction.

Idea. Greedily form a base subset <=target. Its selected weights become negative removal steps; the others become positive addition steps. Shuffle these signed corrections and run packed subset DP inside [-W,W]. Each step is used at most once.

```
bool windowSubset(const vector<ll>& a, ll target, int W,
                  int trials, mt19937_64& rng) {
    assert(target >= 0 && W >= 0 && W <= (INT_MAX-1)/2 && trials >= 0);
    if (!target) return true;
    vector<ll> delta; ll base = 0;
    for (ll x : a) {
        assert(x >= 0);
        if (x <= target-base) { base += x; delta.push_back(-x); }
        else delta.push_back(x);
    }
    ll goal = target-base;
    if (!goal) return true;
    if (goal > W) return false;
    int size = 2*W+1;
    while (trials--) {
        shuffle(delta.begin(), delta.end(), rng);
        int words = (size+63LL)/64;
        vector<ull> dp(words); dp[W/64] = 1ULL << (W%64);
        for (ll x : delta) {
            if (x > 2LL*W || x < -2LL*W) continue;
            int q = abs(x)/64, b = abs(x)%64;
            if (x > 0) {
                for (int i = words-1; i >= q; --i) {
                    ull v = dp[i-q] << b;
                    if (b && i > q) v |= dp[i-q-1] >> (64-b);
                    dp[i] |= v;
                }
            } else if (x < 0) {
                for (int i = 0; i+q < words; ++i) {
                    ull v = dp[i+q] >> b;
                    if (b && i+q+1 < words) v |= dp[i+q+1] << (64-b);
                    dp[i] |= v;
                }
            }
            if (size%64) dp.back() &= (1ULL << (size%64))-1;
        }
        int at = W+(int)goal;
        if (dp[at/64] >> (at%64) & 1) return true;
    }
    return false;
}
```

Complexity. O(t n ceil((2W+1)/64)) time, O(n+W/64) memory. W controls resources, not a proven success probability. With W at least the sum of all weights, no state is clipped, but the speed advantage disappears.

Pitfalls. Weights and target must be nonnegative signed 64-bit values. The code requires 0<=W<=(INT_MAX-1)/2, and available memory imposes a much smaller limit. Large useful corrections can leave the window and return later; clipping loses them. Repetition does not fix every too-small window.

Example. `windowSubset({4,7,9}, 11, 20, 10, rng)` is true. For `{6,10}`, target 10 and W=3, the greedy residual is outside the window: false despite a solution.

Tests. Packed operations versus independent byte DP using identical shuffle seeds, full-window equivalence to exact subset DP, no false positives against exact DP, 64-bit shift boundaries, zeros and LLONG_MAX.

Sources. Knapsack, Subset Sum and the (max,+) Convolution.

9. Schoning randomized 3-SAT

BOUNDED ERROR / VERIFIED WITNESS

Purpose. Find a satisfying assignment for clauses of at most three literals. A returned vector contains 0/1 variable values; nullopt means no witness was found within the budget.

Idea. Start from a uniform random assignment. Find an unsatisfied clause and flip a uniformly chosen literal's variable. After 3n flips, restart. An empty clause is an immediate contradiction; an empty formula is satisfied.

```
optional<vector<int>> schoning(int n, const vector<vector<int>>& clauses,
                               int restarts, mt19937_64& rng) {
    assert(n >= 0 && restarts >= 0);
    for (const auto& c : clauses) {
        assert(c.size() <= 3);
        if (c.empty()) return nullopt;
        for (int lit : c) assert(lit != 0 && abs((ll)lit) <= n);
    }
    if (clauses.empty()) return vector<int>(n);
    while (restarts--) {
        vector<int> a(n);
        for (int& x : a) x = pick(rng, 0, 1);
        for (ll step = 0; step <= 3LL*n; ++step) {
            int bad = -1;
            for (int i = 0; i < (int)clauses.size(); ++i) {
                bool ok = false;
                for (int lit : clauses[i])
                    ok |= a[abs(lit)-1] == (lit > 0);
                if (!ok) { bad = i; break; }
            }
            if (bad == -1) return a;
            if (step == 3LL*n) break;
            const auto& c = clauses[bad];
            int lit = c[pick(rng, 0, c.size()-1)];
            a[abs(lit)-1] ^= 1;
        }
    }
    return nullopt;
}
```

Complexity. O(R n m) time for R restarts, n variables and m clauses; O(n) extra memory. For satisfiable formulas, success per restart is at least (3/4)^n, so miss probability is at most (1-(3/4)^n)^R. This can require exponentially many restarts.

Pitfalls. Literal +(v+1) means x[v]; -(v+1) means not x[v]. Literal 0 is invalid. Clauses contain at most three literals. Timeout cannot certify UNSAT. The implementation deliberately scans clauses; maintaining occurrence lists is a problem-specific optimization.

Example. `schoning(2, {{1,2},{-1,2}}, 100, rng)` finds an assignment with x[1]=1. Check every clause before using a witness.

Tests. Exhaustive truth-table checks for 500 small random formulas, unit clauses, contradictions, duplicates, tautologies, empty clause/formula, zero restarts, and witness validation.

Sources. Schuler, Schoning, Watanabe: An Improved Randomized Algorithm for 3-SAT.

10. Consecutive-point Lagrange interpolation

EXACT UNDER POLYNOMIAL CONTRACT

Purpose. Evaluate a polynomial f at a huge or negative x from y[i]=f(i), 0<=i<n. Useful for polynomial sums and counting formulas with a proven degree bound.

Idea. Lagrange's i-th term is y[i] times the product of (x-j) over j!=i, divided by i! (n-1-i)! (-1)^(n-1-i). Prefix and suffix products make each numerator O(1); inverse factorials handle denominators.

```
const long long LAGRANGE_MOD=1000000007;
long long lagPow(long long a,long long e) {
    long long r=1;
    for(;e;e>>=1,a=a*a%LAGRANGE_MOD) if(e&1) r=r*a%LAGRANGE_MOD;
    return r;
}
long long lagrangeConsecutive(vector<long long> y,long long x) {
    int n=y.size(); if(!n) return 0;
    x=(x%LAGRANGE_MOD+LAGRANGE_MOD)%LAGRANGE_MOD;
    for(auto& v:y) v=(v%LAGRANGE_MOD+LAGRANGE_MOD)%LAGRANGE_MOD;
    if(x<n) return y[x];
    vector<long long> pre(n+1,1),suf(n+1,1),fact(n),ifact(n);
    for(int i=0;i<n;++i) pre[i+1]=pre[i]*(x-i+LAGRANGE_MOD)%LAGRANGE_MOD;
    for(int i=n-1;i>=0;--i) suf[i]=suf[i+1]*(x-i+LAGRANGE_MOD)%LAGRANGE_MOD;
    fact[0]=1;for(int i=1;i<n;++i)fact[i]=fact[i-1]*i%LAGRANGE_MOD;
    ifact[n-1]=lagPow(fact[n-1],LAGRANGE_MOD-2);
    for(int i=n-1;i;--i)ifact[i-1]=ifact[i]*i%LAGRANGE_MOD;
    long long ans=0;
    for(int i=0;i<n;++i) {
        long long term=y[i]*pre[i]%LAGRANGE_MOD*suf[i+1]%LAGRANGE_MOD;
        term=term*ifact[i]%LAGRANGE_MOD*ifact[n-1-i]%LAGRANGE_MOD;
        if((n-1-i)&1) ans=(ans-term+LAGRANGE_MOD)%LAGRANGE_MOD;
        else ans=(ans+term)%LAGRANGE_MOD;
    }
    return ans;
}
```

Complexity. O(n + log P) time and O(n) memory per evaluation; P = 1000000007. Reuses the main notebook's existing implementation without modifying it.

Pitfalls. Prove degree(f)<n and require n<P. Samples are specifically at 0,1,...,n-1; use the next entry for arbitrary x-coordinates. x and y are interpreted modulo P. A sampled x returns y[x] directly. Empty input returns 0 by implementation convention, not a uniquely determined polynomial.

Example. `lagrangeConsecutive({0,1,4}, 10) == 100`. For sum(1..x), samples `{0,1,3}` describe a degree-2 polynomial.

Tests. Existing 4,000 random-polynomial tests against direct evaluation; additional empty/constant cases, negative samples, LLONG_MIN/MAX, sample indices, and x congruent to a sample modulo P.

Sources. Local derivation; Existing tests.

11. General Lagrange interpolation

EXACT UNDER POLYNOMIAL CONTRACT

Purpose. Evaluate f(at) from arbitrary sample pairs (x[i], y[i]), assuming degree(f)<n. Coordinates may be unsorted, negative, or widely spaced. All arithmetic is modulo P=1000000007.

Idea. The i-th basis polynomial is the product of (at-x[j])/(x[i]-x[j]) over j!=i. It equals 1 at x[i] and 0 at the other sample points. Sum y[i] times these basis polynomials; modular inverses implement division.

```
optional<ll> lagrangeGeneral(vector<ll> x, vector<ll> y, ll at) {
    if (x.empty() || x.size() != y.size()) return nullopt;
    int n = x.size(); at = hnorm(at);
    for (ll& v : x) v = hnorm(v);
    for (ll& v : y) v = hnorm(v);
    ll ans = 0;
    for (int i = 0; i < n; ++i) {
        ll num = y[i], den = 1;
        for (int j = 0; j < n; ++j) if (j != i) {
            if (x[i] == x[j]) return nullopt;
            num = num * hnorm(at-x[j]) % HMOD;
            den = den * hnorm(x[i]-x[j]) % HMOD;
        }
        ans = (ans + num*hpow(den, HMOD-2)) % HMOD;
    }
    return ans;
}
```

Complexity. O(n^2 + n log P) time per evaluation, with n modular inverses. For this fixed prime, this is quadratic in n. O(n) memory for normalized input copies. Prefer the previous O(n+log P) version when x[i]=i.

Pitfalls. x and y must have equal nonzero length. x residues must be pairwise distinct modulo P: 1 and P+1 collide. Invalid input returns nullopt, even for repeated pairs with identical y. A valid zero result is a present optional containing 0. Normalize before subtraction to avoid overflow for extreme signed inputs. Prove the degree bound; more than one higher-degree polynomial can fit the same samples.

Example. `lagrangeGeneral({-1,2,4}, {1,4,16}, 3)` contains 9 (f(x)=x^2). Check the optional before dereferencing. A query at a sample point returns its normalized y; duplicate validation still runs.

Tests. Empty/mismatched arrays, singleton/zero polynomials, repeated coordinates modulo P, sample-point queries, LLONG_MIN/MAX coordinates and values, permutation invariance, and 1,500 random polynomials checked by direct Horner evaluation and the consecutive-point implementation.

Sources. Lagrange basis derivation; General interpolation tests.

12. Connected subsets with one centroid

EXACT / COMPLETE SOLUTION / 1-INDEXED

Purpose. Count connected vertex subsets with exactly one centroid, modulo 998244353. Supplied complete solution; formatting only. Includes its own main and setup.

Idea. dp1[v][k] counts k-vertex subsets containing v on its child side; dp2[v][k] counts those containing its parent on the other side. Subtract equal-size pairs across edges from the total: each two-centroid subtree has one balanced edge.

```
#include <bits/stdc++.h>
using namespace std;
constexpr long long mod = 998244353;
int t, n, si[3005];
long long dp1[3005][3005], dp2[3005][3005], ans;
vector<int> g[3005];
void dfs1(int v, int fa) {
    for (int i = 0; i <= si[v]; i++) dp1[v][i] = dp2[v][i] = 0;
    si[v] = 1, dp1[v][1] = 1;
    for (int i : g[v]) {
        if (i == fa) continue;
        dfs1(i, v);
        for (int j = si[v]; ~j; j--) for (int k = 1; k <= si[i]; k++)
            dp1[v][j+k] = (dp1[v][j+k] + dp1[v][j]*dp1[i][k]) % mod;
        si[v] += si[i];
    }
    dp1[v][0] = 1;
    for (int i = 1; i <= si[v]; i++) ans += dp1[v][i];
}
void dfs2(int v, int fa) {
    for (int i = 1; i <= si[v]; i++) ans = (ans - dp1[v][i]*dp2[v][i]) % mod;
    for (int i : g[v]) {
        if (i == fa) continue;
        for (int j = 1; j <= si[i]; j++) dp2[i][j] = dp2[v][j-1];
        for (int j : g[v]) {
            if (j == i || j == fa) continue;
            for (int k = si[i]; ~k; k--)
                for (int l = min(si[j], si[i]-k); l; l--)
                    dp2[i][k+l] = (dp2[i][k+l] + dp2[i][k]*dp1[j][l]) % mod;
        }
        dp2[i][0] = 1, dfs2(i, v);
    }
    g[v].clear();
}
int main() {
    cin >> t;
    while (t--) {
        scanf("%d", &n);
        for (int i = 1, x, y; i < n; i++)
            scanf("%d%d", &x, &y), g[x].emplace_back(y), g[y].emplace_back(x);
        ans = 0, dfs1(1, 0), dp2[1][0] = 1, dfs2(1, 0);
        printf("%lld\n", (ans+mod) % mod);
    }
    return 0;
}
```

Complexity. O(n^2) time. For sibling sizes a,b, their rerooting loops cost O(a min(a,b)) <= O(ab). Each vertex pair belongs to different child subtrees only at its LCA, so these products sum to O(n^2). Downward merges and subtree-size scans also total O(n^2). The two fixed DP arrays occupy about 145 MB (138 MiB).

Pitfalls. 1-indexed tree, 1<=n<=3000. Recursive DFS needs sufficient stack. Keep the reset order and g[v].clear(). Do not disable stream synchronization: cin and scanf are mixed. Centroid means component-size balance, not distance center.

Example. Input: `1`, then `3`, then edges `1 2` and `2 3` (one item/edge per line). Output: `4` (three singletons and the full path).

Tests. Exhaustive labeled trees through n=5, random small trees versus connected-subset enumeration with direct centroid checks, and paths/stars at n=3000 interleaved with tiny cases to check resets.

Sources. User-supplied solution; centroid characterization, Lemma 3.2; black-box tests.